DATA STRUCTURES AND ALGORITHMS

LECTURE 1

Lect. PhD. Oneț-Marian Zsuzsanna

Babeș - Bolyai University
Computer Science and Mathematics Faculty

2025 - 2026

Overview

- Course organization
- Abstract data types and data structures
- Pseudocode conventions
- Algorithm analysis

Course Organization I

- Guiding teachers
  - Lect. PhD. Oneț-Marian Zsuzsanna (zsuzsanna.onet@ubbcluj.ro)
  - Asist. PhD. Albu Alexandra (alexandra.albu@ubbcluj.ro)
  - Asist. PhD. Briciu Anamaria (anamaria.briciu@ubbcluj.ro)
  - PhD. Student Chelaru Ioana (ioana.chelaru@ubbcluj.ro)

Course Organization II

- Activities
  - **Lecture:** 2 hours / week
  - **Seminar:** 1 hour / week
  - **Lab:** 1 hour / week
    - Please use your *ubbcluj* email address for communication (and make sure the email contains your name and (semi)group). Alternatively, you can use MS Teams private chat.

Course Organization III

- Grading
  - Written exam (**W**) - will be in the exam session
  - Lab grade (**LG**) - average weighted grade of the presented lab assignments (more details in the LabRules.pdf document)
  - Seminar bonus (**B**) - maximum 0.5 points bonus, received for the seminar activity.
  - The final grade is computed as:

$$G = 0.7 * W + 0.3 * LG + B$$

- **To pass the exam, W and G have to be $\geq$ 5 (no rounding)**
- In the retake session only the written exam can be repeated, and grade G will be computed in the same way as in the regular session.

Platforms/Resources used during the semester

- MS Teams - will be used for Lectures, Seminars and Labs. All the materials (lecture slides, lab requirements, etc.) will be uploaded in the General channel at the Files section. Join the team with code **ipbreoo**.

- Moodle - used for optional quizes.
  - After every lecture you will have a quiz on Moodle, where you can test how well you understood the lecture. The quiz is optional, and will be available for one week.
  - Link to Moodle: https://moodle.cs.ubbcluj.ro/ - join the DSA - IA + II - 2025 - 2026 course.

- Optional Extra reading - on MS Teams.
  - Some lectures will be accompanied by an Extra Reading document as well, where I will include extra information, real-life examples, interesting problems, etc. Reading them is entirely optional.

Rules I

- Attendance is compulsory for the laboratory and the seminar activity. You need at least
  - 6 attendances from the 7 labs
  - 5 attendances from the 7 seminars.

- **Unless you have the required number of attendances, you cannot participate in the written exam, neither in the regular nor in the retake session!**

- The General channel in Files → Class material folder contains a document (AttendanceDocLink.pdf) with a link to a Google Sheets document where you can check your attendance situation using your code from AcademicInfo. In this document you will also see your lab assignments, lab grades and seminar activity.

Rules II

- You have to come to the seminar with your group and to the lab with your semi-group (we will consider the official student lists for the groups).
- If you want to *permanently* switch from one (semi-)group to another, you have to find a person in the other (semi-)group who is willing to switch with you and announce your lab/seminar teacher about the switch in the first two weeks of the semester.
- One seminar and one laboratory attendance can be recovered with another group, within the two weeks allocated for the seminar/laboratory, with the explicit agreement of the seminar/laboratory teacher.

Rules III

- In case of illness, absences will be motivated by the lab/seminar teacher, based on a medical certificate. Medical certificates have to be presented in the first seminar/lab after the absence, certificates presented later than that will not be accepted.

Bibliography I

- T. CORMEN, C. LEISERSON, R. RIVEST, C. STEIN, *Introduction to algorithms*, Third Edition, The MIT Press, 2009
- S. SKIENA, *The algorithm design manual*, Second Edition, Springer, 2008
- N. KARUMANCHI, *Data structures and algorithms made easy*, CareerMonk Publications, 2016
- M. A WEISS, *Data structures and algorithm analysis in Java*, Third Edition, Pearson, 2012

Bibliography II

- R. SEDGEWICK, *Algorithms*, Fourth Editions, Addison-Wesley Publishing Company, 2011
- R. LAFORE, *Data Structures & Algorithms in Java*, Second Edition, Sams Publishing, 2003
- M. A. WEISS *Data structures and problem solving using C++*, Second Edition, Pearson, 2003

Course Objectives

- The study of the concept of **abstract data types** and the most frequently used container abstract data types.
- The study of different **data structures** that can be used to implement these abstract data types and the complexity of their operations.
- What you should learn from this course:
  - to design and implement different applications starting from the use of abstract data types.
  - to process data stored in different data structures.
  - to choose the abstract data type and data structure best suited for a given application.

Abstract Data Types

- What is a data type? Could you give me examples of data types that you know?

Abstract Data Types

- What is a data type? Could you give me examples of data types that you know?
- A *data type* is a set of values (domain) and a set of operations on those values. For example:
  - int
    - set of values are integer numbers from a given interval
    - possible operations: add, subtract, multiply, etc.
  - boolean
    - set of values: true, false
    - possible operations: negation, and, or, xor, etc.
  - String
    - "abc", "text", etc.
    - possible operations: get the length, get a character from a position, get a substring, concatenate two strings, etc.
  - etc.
- We can have built-in data types and user defined data types. How would you define a data type for a rational number?

Abstract Data Types

- An Abstract Data Type (ADT) is a *data type* having the following two properties:
  - the objects from the domain of the ADT are specified independently of their representation
  - the operations of the ADT are specified independently of their implementation

Abstract Data Types - Domain

- The domain of an ADT describes what elements belong to this ADT.
- If the domain is finite, we can simply enumerate them.
- If the domain is not finite, we will use a rule that describes the elements belonging to the ADT.

Abstract Data Types - Interface

- After specifying the domain of an ADT, we need to specify its operations.
- The set of all operations for an ADT is called its *interface*.
- The interface of an ADT contains the *signature* of the operations, together with their input data, results, preconditions and postconditions (but no detail regarding the implementation of the method).
- When talking about ADT we focus on the **WHAT** (it is, it does), not on the **HOW** (it is represented, it is implemented).

ADT - Example I

- Consider the Date ADT (which represents a date from the calendar: year, month and day)
  - **Domain** (what is a valid date?): year is positive, maybe less than 3000, month is between 1 and 12, day is between 1 and maximum number of possible days for the month
  - **Interface** (what can we do with dates?): one possible operation for the Date ADT could be: difference(d1, d2)
    - **Descr:** computes the difference in number of days between two dates
    - **Pre:** d1 and d2 have to be valid Dates, d1 $\leq$ d2
    - **Post:** difference $\leftarrow$ d, d is a natural number and represents the number of days between d1 and d2.
    - **Throws:** and exception if d1 is greater than d2

ADT - Example II

- This information specifies everything we need to use the Date ADT, even if we know nothing about how it is represented or how the operation *difference* is implemented.

Advantages of working with ADTs I

- Assume that you have decided to represent the Date by using 3 integer numbers: one for the year, one for the month and one for the day.
- Assume that you have defined a lot of operations for Date: compute the difference between two dates, check if a date is in the weekend, add a given number of days to a date, etc.
- Assume that you have also defined an ADT to represent a *TimeInterval*, which contains two Dates: the start and the end of the interval.
- Assume that you have implemented a list of operations for *TimeInterval*: check if two intervals overlap, check if an interval includes a specific day, create an interval starting from a date and a number of days, etc.

Advantages of working with ADTs II

- Assume that you have implemented a complex application for scheduling holidays (*TimeIntervals*) and setting up deadlines for tasks (*Date*) and all sort of similar functionalities.

- What happens if now you want to modify the representation of the Date? Instead of storing 3 numbers, you want to store a date as a number with 8 digits, first 4 digits are the year, next two are the month and the last two are the days. How much of your code do you need to change?

Advantages of working with ADTs III

- Well, obviously ADT Date needs to be rewritten, together with all the operations defined for it.
- But, if you worked with Date as an ADT, even if the representation changes and even if the implementation of the operations changes, they still have to respect the interface (ex: the signature and behaviour of the difference operation is still the same). This means that the rest of the application is not affected by the changes.

Advantages of working with ADTs IV

- *Abstraction* is defined as the separation between the specification of an object (its domain and interface) and its implementation.

- *Encapsulation* - Encapsulation means that we hide the representation and implementation details inside the implementation (most time, inside the class). Abstraction provides a promise that any implementation of an ADT will belong to its domain and will respect its interface. And this is all that is needed to use an ADT.

Advantages of working with ADTs V

- *Localization of change* - any code that uses an ADT is still valid if the ADT changes (because no matter how it changes, it still has to respect the domain and interface).
- *Flexibility* - an ADT can be implemented in different ways, but all these implementation have the same interface. Switching from one implementation to another can be done with minimal changes in the code.

Container ADT I

- A *container* is a collection of data, in which we can add new elements and from which we can remove elements.
- Different containers are defined based on different properties:
  - do the elements need to be unique?
  - do the elements have positions assigned?
  - can any element be accessed or just some specific ones?
  - do we store simple elements or key - value pairs?

Container ADT II

- A container should provide at least the following operations (the interface of the container should contain operations for):
  - *creating* an empty container
  - *adding* a new element to the container
  - *removing* an element from the container
  - returning the *number of elements* in the container
  - providing *access to the elements* from the container (usually using an *iterator*)

Container vs. Collection

- Python - Collections
- C++ - Containers from STL
- Java - Collections framework and the Apache Collections library
- .Net - System.Collections framework
- In the following, in this course we will use the term **container**.

- During the semester we are going to talk about many different containers, but there are two containers that you are already familiar with from Python:

- During the semester we are going to talk about many different containers, but there are two containers that you are already familiar with from Python: *list* and *dict*.

- Moreover, you know them and you have used them as ADT!

- During the semester we are going to talk about many different containers, but there are two containers that you are already familiar with from Python: *list* and *dict*.
- Moreover, you know them and you have used them as ADT!
  - Do you know how a *list* or a *dict* is represented?
  - Do you know how their operations are implemented?
  - Did not knowing these stop you from using them?
- While the *list* and the *map* (this is what we will use for *dict*) are indeed the two most popular containers, during the semester we will talk about several other containers: Bag, Set, Stack, Queue, PriorityQueue, Matrix, Multimap, Tree.

Why container abstract data types?

- There are several different container Abstract Data Types, so choosing the most suitable one is an important step during application design.
- When choosing the suitable ADT we are not interested in the implementation details of the ADT (yet).
- Most high-level programming languages usually provide implementations for different Abstract Data Types.
  - In order to be able to use the right ADT for a specific problem, we need to know their domain and interface.

Example

- Assume that you have to write an application to handle a numeral system similar to the Roman Numerals. This application needs to be able to define the mappings for the numbers (for example Z to represent 2000) and convert them into Arabic numerals and vice versa.
- In order to implement this application you will often need to find the number corresponding to a given letter and vice versa.
- What container would you use?

Data Structures I

- The domain of data structures studies how we can store and access data.
- A data structure can be:
  - Static: the size of the data structure is fixed. Such data structures are suitable if it is known that a fixed number of elements need to be stored.
  - Dynamic: the size of the data structure can grow or shrink as needed by the number of elements.

Data Structures II

- For every container ADT we will discuss several possible data structures that can be used for the implementation. For every possibility we will discuss the advantages and disadvantages of using the given data structure. We will see that, in general, we cannot say that there is one single *best* data structure for a given container.

Why?

- Why do we need to discuss how container Abstract Data Types can be implemented (and implement them during the labs) if they are already implemented in most programming languages?

Why?

- Why do we need to discuss how container Abstract Data Types can be implemented (and implement them during the labs) if they are already implemented in most programming languages?
  - Implementing these ADT will help us understand better how they work (we cannot use them efficiently, if we do not know how they are implemented)
    - For example, in Python the following instruction has a different complexity depending on whether *cont* is a list or a dict

```
if elem in cont:
    print("Found")
```

Why?

- To learn to create, implement and use ADTs for situations when:
  - we work in a programming language where they are not readily implemented.
  - we need an ADT which is not part of the standard ones, but might be similar to them.

Pseudocode I

- The aim of this course is to give a general description of data structures, one that does not depend on any programming language - so we will use the *pseudocode* language to describe the algorithms.
- Our algorithms written in pseudocode will consist of two type of instructions:
  - standard instructions (assignment, conditional, repetitive, etc.)
  - non-standard instructions (written in plain English to describe parts of the algorithm that are not developed yet). These non-standard instructions will start with @.

Pseudocode II

- One line comments in the code will be denoted by //
- For reading data we will use the standard instruction **read**
- For printing data we will us the standard instruction **print**
- For assignment we will use $\leftarrow$
- For testing the equality of two variables we will use $=$

Pseudocode III

- Conditional instruction will be written in the following way (the *else* part can be missing):

```
if condition then
    @instructions
else
    @instructions
end-if
```

Pseudocode IV

- The *for* loop (loop with a known number of steps) will be written in the following way:

```
for i ← init, final, step execute
    @instructions
end-for
```

- *init* - represents the initial value for variable i
- *final* - represents the final value for variable i
- *step* - is the value added to i at the end of each iteration. *step* can be missing, in this case it is considered to be 1.

Pseudocode V

- The *while* loop (loop with an unknown number of steps) will be written in the following way:

while condition **execute**
  @instructions
end-while

Pseudocode VI

- Subalgorithms (subprograms that do not return a value) will be written in the following way:

subalgorithm name(formal parameter list) **is:**
  @instructions - subalgorithm body
end-subalgorithm

- The subalgorithm can be called as:

name (actual parameter list)

Pseudocode VII

- Functions (subprograms that return a value) will be written in the following way:

function name (formal parameter list) **is:**
  @instructions - function body
  name ← v *//syntax used to return the value v*
end-function

- The function can be called as:

result ← name (actual parameter list)

Pseudocode VIII

- If we want to define a variable i of type Integer, we will write: i : *Integer*
- If we want to define an array a, having elements of type T, we will write: $a : T[]$
  - If we know the size of the array, we will use: $a : T[Nr]$ - indexing is done from 1 to Nr
  - If we do not know the size of the array, we will use: $a : T[]$ - indexing is done from 1

Pseudocode IX

- Pseudocode is not object-oriented, we will work with structures (records), not classes.
- A struct (record) will be defined as:

Array:
n: Integer
elems: T[]

- The above struct consists of 2 fields: *n* of type Integer and an array of elements of type T called *elems*
- Having a variable *var* of type Array, we can access the fields using . (dot):
  - var.n
  - var.elems
  - var.elems[i] - the i-th element from the array

Pseudocode X

- For denoting pointers (variables whose value is a memory address) we will use $\uparrow$:
  - p: $\uparrow$ Integer - p is a variable whose value is the address of a memory location where an Integer value is stored.
  - The value from the address denoted by p is accessed using $[p]$
- Allocation and de-allocation operations will be denoted by:
  - allocate(p)
  - free(p)
- We will use the special value NIL to denote an invalid address

Specifications I

- An operation will be specified in the following way:
  - **pre:** - the preconditions of the operation
  - **post:** - the postconditions of the operation
  - **throws:** - exceptions thrown (optional - not every operation can throw an exception)
- When using the name of a parameter in the specification we actually mean its value.
- Having a parameter i of type T, we will denote by $i \in T$ the condition that the value of variable i belongs to the domain of type T.

Specifications II

- The value of a parameter can be changed during the execution of a function/subalgorithm. To denote the difference between the value before and after execution, we will use the ' (apostrophe).

- For example, the specification of an operation *decrement*, that decrements the value of a parameter x (x : *Integer*) will be:
  - **pre:** x $\in$ Integer
  - **post:** x' = x - 1

Generic Data Types I

- We will consider that the elements of a container ADT are of a generic type: *TElem*
- The interface of the *TElem* contains the following operations:
  - assignment $(e_1 \leftarrow e_2)$
    - **pre:** $e_1, e_2 \in TElem$
    - **post:** $e_1' = e_2$
  - equality test $(e_1 = e_2)$
    - **pre:** $e_1, e_2 \in TElem$
    - **post:**

$$equal \leftarrow \begin{cases} True, & \text{if } e_1 \text{ } equals \text{ } e_2 \\ False, & \text{otherwise} \end{cases}$$

Generic Data Types II

- When the values of a data type can be compared or ordered based on a relation, we will use the generic type: *TComp*.
- Besides the operations from TElem, TComp has an extra operation that compares two elements:
  - compare(e_1, e_2)
    - **pre:** $e_1, e_2 \in TComp$
    - **post:**

$$compare \leftarrow \begin{cases} true & \text{if } e_1 <= e_2 \\ false & \text{if } e_1 > e_2 \end{cases}$$

- For simplicity, sometimes instead of calling the *compare* function, we will use the notations $e_1 \leq e_2$, $e_1 = e_2$, $e_1 > e_2$

Generic Data Types III

- **Obs:** it is important to have a convention about what the relation/comparison function returns if the elements are equal, because this influences the implementation of operations using such a relation. During this course, we will assume that in case of equality the relation returns TRUE. You can interpret it in the following way: *the relation returns true if the two elements are in the right order - they do not need to be interchanged*.

The RAM model I

- Analyzing an algorithm usually means predicting the resources (time, memory) the algorithm requires. In order to do so, we need a hypothetical computer model, called *RAM* (random-access machine) model.
- In the RAM model:
  - Each simple operation (+, -, *, /, =, if, function call) takes one time step/unit.
  - We have fixed-size integers and floating point data types.
  - Loops and subprograms are *not* simple operations and we do not have special operations (ex. sorting in one instruction).
  - Every memory access takes one time step and we have an infinite amount of memory.

The RAM model II

- The RAM is a very simplified model of how computers work, but in practice it is a good model to understand how an algorithm will perform on a real computer.

- Under the RAM model we measure the run time of an algorithm by counting the number of steps the algorithm takes on a given input instance. The number of steps is usually a function that depends on the size of the input data.

```
subalgorithm something(n) is:
//n is an Integer number
    rez ← 0
    for i ← 1, 2*n execute
        sum ← 0
        for j ← 1, n execute
            sum ← sum + j
        end-for
        rez ← rez + sum
    end-for
    print rez
end-subalgorithm
```

- How many steps does the above subalgorithm take?

```
subalgorithm something(n) is:
//n is an Integer number
    rez ← 0
    for i ← 1, 2*n execute
        sum ← 0
        for j ← 1, n execute
            sum ← sum + j
        end-for
        rez ← rez + sum
    end-for
    print rez
end-subalgorithm
```

- How many steps does the above subalgorithm take?
- T(n) = 1 + 2 * n * (1 + n + 1) + 1 = $2n^2 + 4n + 2$

Order of growth

- We are not interested in the exact number of steps for a given algorithm, we are interested in its *order of growth* (i.e., how does the number of steps change if the value of n increases)

- We will consider only the leading term of the formula without constants (for example n^2), because the other terms are relatively insignificant for large values of n (basically, $2n^2 + 4n + 2$ grows as fast as n^2 when n goes to infinity, so it is enough to consider n^2).

O-notation I

O-notation

For a given function $g(n)$ we denote by $O(g(n))$ the set of functions:

$O(g(n)) = \{f(n)$: there exist positive constants c and n_0 s. t. $0 \leq f(n) \leq c \cdot g(n)$ for all $n \geq n_0\}$

- The O-notation provides an *asymptotic upper bound* for a function: for all values of n (to the right of n_0) the value of the function $f(n)$ is on or below $c \cdot g(n)$.
- We will use the notation $f(n) = O(g(n))$ or $f(n) \in O(g(n))$.

O-notation II

- Graphical representation:

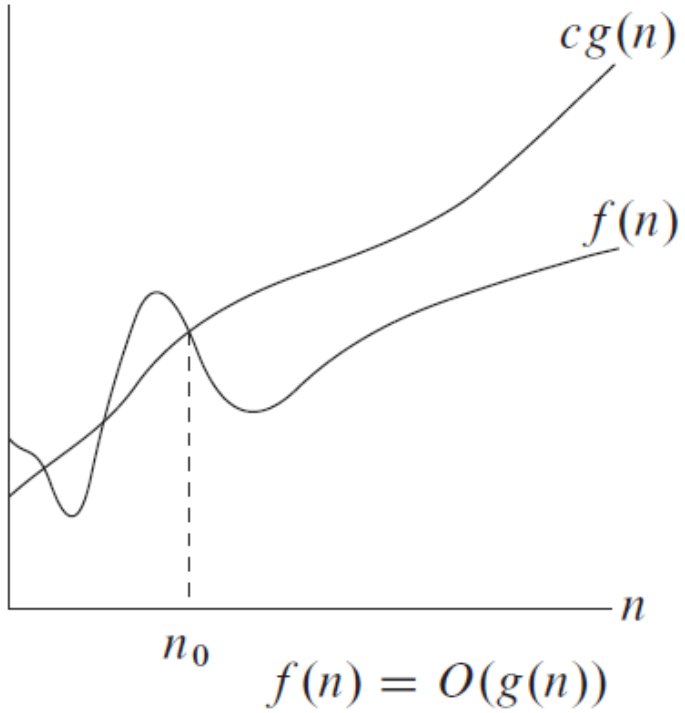

Figure taken from Corman et. al: Introduction to algorithms, MIT Press, 2009

O-notation III

Alternative definition

$$f(n) \in O(g(n)) \text{ if } \lim_{n \to \infty} \frac{f(n)}{g(n)}$$

is a constant (but not ∞).

- Consider, for example, $T(n) = n^2 + 2n + 2$:
  - $T(n) = O(n^2)$ because $T(n) \leq c * n^2$ for c $= 2$ and n ≥ 3
  - $T(n) = O(n^3)$ because

$$\lim_{n \to \infty} \frac{T(n)}{n^3} = 0$$

Ω-notation I

Ω-notation

For a given function $g(n)$ we denote by $\Omega(g(n))$ the set of functions:

$\Omega(g(n)) = \{f(n) :$ there exist positive constants c and n_0 s. t.
$0 \leq c \cdot g(n) \leq f(n)$ for all $n \geq n_0\}$

- The Ω-notation provides an *asymptotic lower bound* for a function: for all values of n (to the right of n_0) the value of the function $f(n)$ is on or above $c \cdot g(n)$.
- We will use the notation $f(n) = \Omega(g(n))$ or $f(n) \in \Omega(g(n))$.

Ω-notation II

- Graphical representation:

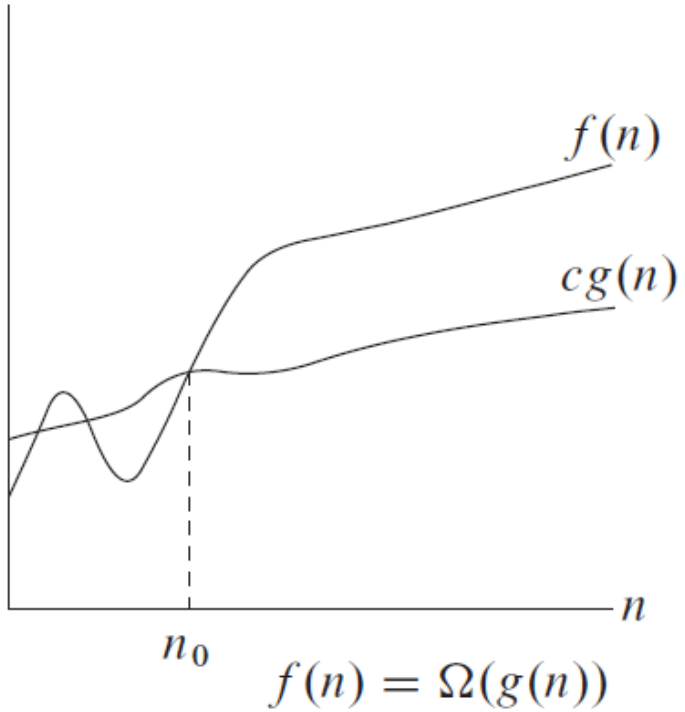

Figure taken from Corman et. al: Introduction to algorithms, MIT Press, 2009

Ω-notation III

Alternative definition

$$f(n) \in \Omega(g(n)) \text{ if } \lim_{n\to\infty} \frac{f(n)}{g(n)}$$

is ∞ or a nonzero constant.

- Consider, for example, $T(n) = n^2 + 2n + 2$:
  - $T(n) = \Omega(n^2)$ because $T(n) \geq c * n^2$ for c = 0.5 and n ≥ 1
  - $T(n) = \Omega(n)$ because

$$\lim_{n\to\infty} \frac{T(n)}{n} = \infty$$

Θ-notation I

Θ-notation

For a given function $g(n)$ we denote by $\Theta(g(n))$ the set of functions:

$$\Theta(g(n)) = \{f(n) : \text{there exist positive constants } c_1, c_2 \text{ and } n_0 \text{ s. t. } 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ for all } n \geq n_0\}$$

- The Θ-notation provides an *asymptotically tight bound* for a function: for all values of n (to the right of n_0) the value of the function $f(n)$ is between $c_1 \cdot g(n)$ and $c_2 \cdot g(n)$.
- We will use the notation $f(n) = \Theta(g(n))$ or $f(n) \in \Theta(g(n))$.

Θ-notation II

- Graphical representation:

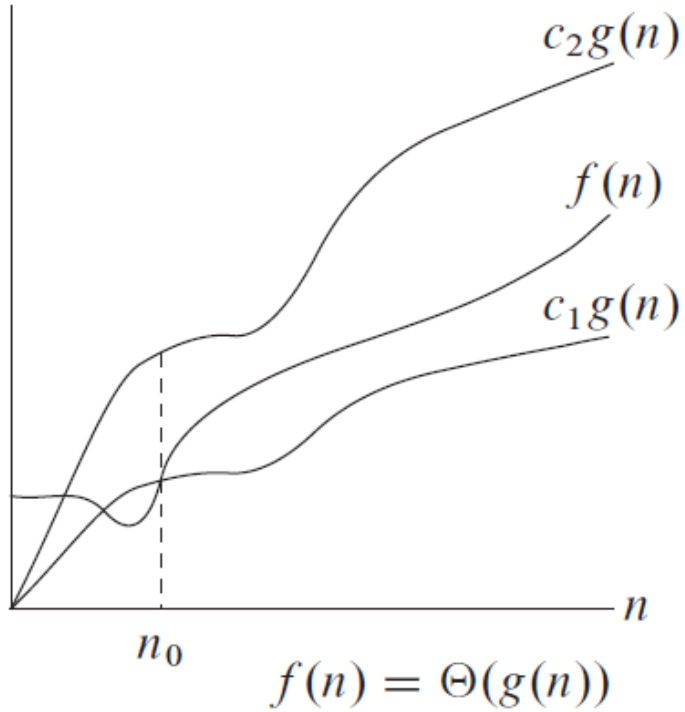

Figure taken from Corman et. al: Introduction to algorithms, MIT Press, 2009

Θ-notation III

Alternative definition

$$f(n) \in \Theta(g(n)) \text{ if } \lim_{n\to\infty} \frac{f(n)}{g(n)}$$

is a nonzero constant (and not ∞).

- Consider, for example, $T(n) = n^2 + 2n + 2$:
  - $T(n) = \Theta(n^2)$ because $c_1 * n^2 \leq T(n) \leq c_2 * n^2$ for $c_1 = 0.5$, $c_2 = 2$ and n ≥ 3.
  - $T(n) = \Theta(n^2)$ because

$$\lim_{n\to\infty} \frac{T(n)}{n^2} = 1$$

Best Case, Worst Case, Average Case I

- Think about an algorithm that finds the sum of all even numbers in an array. How many steps does the algorithm take for an array of length n?
- Think about an algorithm that finds the first occurrence of a given number in an array. How many steps does the algorithm take for an array of length n?

Best Case, Worst Case, Average Case II

- For the second problem the number of steps taken by the algorithm does not depend just on the length of the array, it depends on the exact values from the array as well.
- For an array of fixed length n, execution of the algorithm can stop after:
  - verifying the first number - if it is the one we are looking for
  - verifying the first two numbers - if the first is not the one we are looking for, but the second is
  - ...
  - verifying all n numbers - first $n-1$ are not the one we are looking for, but the last is, or none of the numbers is the one we are looking for

Best Case, Worst Case, Average Case III

- For such algorithms we will consider three cases:
  - Best - Case (BC) - the best possible case, where the number of steps taken by the algorithm is the minimum that is possible
  - Worst - Case (WC) - the worst possible case, where the number of steps taken by the algorithm is the maximum that is possible
  - Average - Case (AC) - the average of all possible cases.
- Best and Worst case complexity is usually computed by inspecting the code. For our example we have:
  - Best case: $\Theta(1)$ - just the first number is checked, no matter how large the array is.
  - Worst case: $\Theta(n)$ - we have to check all the numbers

Best Case, Worst Case, Average Case IV

- For computing the average case complexity we have a formula:

$$\sum_{I \in D} P(I) \cdot E(I)$$

- where:
  - D is the domain of the problem, the set of every possible input that can be given to the algorithm.
  - I is one input data
  - $P(I)$ is the probability that we will have I as an input
  - $E(I)$ is the number of operations performed by the algorithm for input I

Best Case, Worst Case, Average Case V

- For our example D would be the set of all possible arrays with length n
- Every I would represent a subset of D:
  - One I represents all the arrays where the first number is the one we are looking for
  - One I represents all the arrays where the first number is not the one we are looking for, but the second is
  - ...
  - One I represents all the arrays where the first $n-1$ elements are not the one we are looking for but the last one is
  - One I represents all the arrays which do not contain the element we are looking for
- P(I) is usually considered equal for every I, in our case $\frac{1}{n+1}$

$$T(n) = \frac{1}{n+1}\sum_{i=1}^{n} i + \frac{n}{n+1} = \frac{n\cdot(n+1)}{2\cdot(n+1)} + \frac{n}{n+1} \in \Theta(n)$$

Best Case, Worst Case, Average Case VI

- When we have best case, worst case and average case complexity, we will report the maximum one (which is the worst case), but if the three values are different, the total complexity is reported with the O-notation.

- For our example we have:
  - Best case: $\Theta(1)$
  - Worst case: $\Theta(n)$
  - Average case: $\Theta(n)$
  - Total (overall) complexity: $O(n)$

- **Obs**: For best, worst and average case we will always use the Θ notation.

Algorithm Analysis for Recursive Functions

- How can we compute the time complexity of a recursive algorithm?

Recursive Binary Search

```
function BinarySearchR (array, elem, start, end) is:
//array - an ordered array of integer numbers
//elem - the element we are searching for
//start - the beginning of the interval in which we search (inclusive)
//end - the end of the interval in which we search (inclusive)
    if start > end then
        BinarySearchR ← False
    end-if
    middle ← (start + end) / 2
    if array[middle] = elem then
        BinarySeachR ← True
    else if elem < array[middle] then
        BinarySearchR ← BinarySearchR(array, elem, start, middle-1)
    else
        BinarySearchR ← BinarySearchR(array, elem, middle+1, end)
    end-if
end-function
```

Recursive Binary Search

- The first call to the *BinarySearchR* algorithm for an ordered array of *nr* elements is:

```
BinarySearchR(array, elem, 1, nr)
```

- How do we compute the complexity of the BinarySearchR algorithm?

Recursive Binary Search

- We will denote the length of the sequence that we are checking at every iteration by n (so $n = end - start$)
- We need to write the recursive formula of the solution

Recursive Binary Search

- We will denote the length of the sequence that we are checking at every iteration by n (so $n = end - start$)
- We need to write the recursive formula of the solution

$$T(n) = \begin{cases} 1, & \text{if } n \leq 1 \\ T(\frac{n}{2}) + 1, & \text{otherwise} \end{cases}$$

Time complexity for BinarySearchR

- We suppose that $n = 2^k$ and rewrite the second branch of the recursive formula:

$$T(2^k) = T(2^{k-1}) + 1$$

- Now we write what the value of $T(2^{k-1})$ is (based on the recursive formula)

$$T(2^{k-1}) = T(2^{k-2}) + 1$$

- Next, we add what the value of $T(2^{k-2})$ is (based on the recursive formula)

$$T(2^{k-2}) = T(2^{k-3}) + 1$$

Time complexity for BinarySearchR

- The last value that can be written is the value of $T(2^1)$

$$T(2^1) = T(2^0) + 1$$

Time complexity for BinarySearchR

- Now we write all these equations together and add them (and we will see that many terms can be simplified, because they appear on the left hand side of an equation and the right hand side of another equation):

$$T(2^k) = T(2^{k-1}) + 1$$
$$T(2^{k-1}) = T(2^{k-2}) + 1$$
$$T(2^{k-2}) = T(2^{k-3}) + 1$$
$$\dots$$
$$T(2^1) = T(2^0) + 1$$
$$\text{---------------------------} +$$
$$T(2^k) = T(2^0) + 1 + 1 + 1 + \dots + 1 = 1 + k$$

- **Obs:** For non-recursive functions adding a $+1$ or not does not influence the result. In case of recursive functions it is important to have another term besides the recursive one.

Time complexity for BinarySearchR

- We started from the notation $n = 2^k$.
- We want to go back to the notation that uses n. If $n = 2^k \Rightarrow k = log_2 n$

$T(2^k) = 1 + k$
$T(n) = 1 + log_2 n \in \Theta(log_2 n)$

Time complexity for BinarySearchR

- We started from the notation $n = 2^k$.
- We want to go back to the notation that uses n. If $n = 2^k \Rightarrow k = log_2 n$

$T(2^k) = 1 + k$
$T(n) = 1 + log_2 n \in \Theta(log_2 n)$

- Actually, if we look at the code from *BinarySearchR*, we can observe that it has a best case (element can be found at the first iteration), so final complexity is $O(log_2 n)$

Another example

- Let's consider the following pseudocode and compute the time complexity of the algorithm:

```
subalgorithm operation(n, i) is:
//n and i are integer numbers, n is positive
    if n > 1 then
        i ← 2 * i
        m ← n/2
        operation(m, i-2)
        operation(m, i-1)
        operation(m, i+2)
        operation(m, i+1)
    else
        write i
    end-if
end-subalgorithm
```

- The first step is to write the recursive formula:

$$T(n) = \begin{cases} 1, & \text{if } n \leq 1 \\ 4 \cdot T(\frac{n}{2}) + 1, & \text{otherwise} \end{cases}$$

- We suppose that $n = 2^k$.

$$T(2^k) = 4 \cdot T(2^{k-1}) + 1$$

- This time we need the value of $4 \cdot T(2^{k-1})$

$$T(2^{k-1}) = 4 \cdot T(2^{k-2}) + 1 \Rightarrow$$
$$4 \cdot T(2^{k-1}) = 4^2 \cdot T(2^{k-2}) + 4$$

- And the value of $4^2 \cdot T(2^{k-2})$

$$4^2 \cdot T(2^{k-2}) = 4^3 \cdot T(2^{k-3}) + 4^2$$

- The last value we can compute is $4^{k-1} \cdot T(2^1)$

$$4^{k-1} \cdot T(2^1) = 4^k \cdot T(2^0) + 4^{k-1}$$

- We write all the equations and add them:

$$T(2^k) = 4 \cdot T(2^{k-1}) + 1$$
$$4 \cdot T(2^{k-1}) = 4^2 \cdot T(2^{k-2}) + 4$$
$$4^2 \cdot T(2^{k-2}) = 4^3 \cdot T(2^{k-3}) + 4^2$$
$$\ldots$$
$$4^{k-1} \cdot T(2^1) = 4^k \cdot T(2^0) + 4^{k-1}$$
$$\overline{\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad\qquad} +$$
$$T(2^k) = 4^k \cdot T(1) + 4^0 + 4^1 + 4^2 + \ldots + 4^{k-1}$$

- $T(1)$ is 1 (first case from recursive formula)

$$T(2^k) = 4^0 + 4^1 + 4^2 + \ldots + 4^{k-1} + 4^k$$

$$\sum_{i=0}^{n} p^i = \frac{p^{n+1} - 1}{p - 1}$$

$$T(2^k) = \frac{4^{k+1} - 1}{4 - 1} = \frac{4^k \cdot 4 - 1}{3} = \frac{(2^k)^2 \cdot 4 - 1}{3}$$

- We started from $n = 2^k$. Let's change back to n

$$T(n) = \frac{4n^2 - 1}{3} \in \Theta(n^2)$$

Summary

- Today we have talked about:
  - What is a container Abstract Data Type.
  - What is a data structure.
  - Pseudocode conventions.
  - Algorithm analysis (time complexity computation)
- Extra reading:
  - Two more topics related to algorithm analysis:
    - Master method - an alternative way of computing complexities for recursive functions.
    - Empirical algorithm analysis